

CEO GENAI Analysis

- a.) I used Chatgpt and Gemini
- b.) I accessed these GenAi using my laptop on Chrome.

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.

c.)

- Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```
- Your program should display this:

```
There are 6 emails to read.

Next email:
  Sender: Boss
  Subject: Never Mind
  Date: 01-03-2025

Next email:
  Sender: Boss
  Subject: Important
  Date: 12-20-2024

There are 4 emails to read.
```

- Test File format:

Command	Description
EMAIL <sender category>,<subject line>,<date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails
- The grader will test your code with a different test file. The test file will begin with the emails in the CEO's Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

I simply used screen shots of the provided examples and prompts. As well as a prompt asking the ai to explain the way it used time complexity in its code. I also made sure to give both ai's the exact same prompts.

Correctness -

Chatgpt: Prioritizes emails by sender category and then by date. Doesn't display anything when 'next' is called on an empty inbox. Assigns any unrecognized sender categories a priority of 1, which could cause incorrect ordering if unknown categories are present

Gemini: Prioritizes emails by sender category then by date, the new emails take priority. Assigns unknown sender categories a priority of 0. Handles all the email, next, read, and count commands.

Execution Time -

Chatgpt: Uses a single linked list, using traversal to access elements at particular positions. Also uses traverse on the list to find the last node during insertion and removal

Gemini: Does not need to traverse the entire heap to find a single element

Space Complexity -

Chatgpt: Uses a vector to store the emails in a contiguous memory. Might use additional unused space whenever the vector grows.

Gemini: has about the space complexity that chatgpt does but uses more memory for pointers and single node allocations.

Maintainability -

Chatgpt: Separates email, heapnode, maxheap, and ceoinbox for four classes. Has more code to maintain, and its linked list implementation adds more complexity.

Gemini: Makes three classes for email maxheap and the ceoinboxsystem. Uses a vector, which simplifies memory management. Also uses descriptive method names like sift up and sift down.

- f.) I will be choosing gemini's code. I believe it uses the time complexity better in the sense that it doesn't need to use traverse to just check or grab an element. Also the fact that Gemini uses less additional memory for pointers in the linked list. The use of vectors also makes it easier to maintain since it removes the need to maintain the memory manually.
- g.) I would add an error handle to make sure that the file given exists, and as well as each email contains valid categories, subjects, and dates. That would help prevent the program from just crashing.

d.) Gemini code

```
e.)#include <iostream>
f.)#include <string>
```

```

g.)#include <sstream>
h.)#include <vector>
i.)#include <iomanip>
j.)
k.)using namespace std;
l.)
m.)// Class representing an individual Email
n.)class Email {
o.)private:
p.)    string senderCategory;
q.)    string subjectLine;
r.)    string dateStr;
s.)    int priorityRank;
t.)    long dateValue; // Numeric value for easy date comparison (YYYYMMDD)
u.)
v.)    // Convert date format MM-DD-YYYY to a comparable integer YYYYMMDD
w.)    long parseDate(const string& d) {
x.)        if (d.length() != 10) return 0;
y.)        int month = stoi(d.substr(0, 2));
z.)        int day = stoi(d.substr(3, 2));
aa.)        int year = stoi(d.substr(6, 4));
bb.)        return (year * 10000L) + (month * 100L) + day;
cc.)    }
dd.)
ee.)    // Map sender category to a priority rank (higher number =
    higher priority)
ff.)    int getPriorityRank(const string& category) {
gg.)        if (category == "Boss") return 5;
hh.)        if (category == "Subordinate") return 4;
ii.)        if (category == "Peer") return 3;
jj.)        if (category == "ImportantPerson") return 2;
kk.)        if (category == "OtherPerson") return 1;
ll.)        return 0;
mm.)    }
nn.)
oo.)    public:
pp.)        Email() {
qq.)            senderCategory = "";
rr.)            subjectLine = "";
ss.)            dateStr = "";
tt.)            priorityRank = 0;
uu.)            dateValue = 0;
vv.)        }
ww.)
xx.)        Email(string category, string subject, string date) {

```

```

yy.)         senderCategory = category;
zz.)         subjectLine = subject;
aaa.)        dateStr = date;
bbb.)        priorityRank = getPriorityRank(category);
ccc.)        dateValue = parseDate(date);
ddd.)    }
eee.)
fff.)        string getSenderCategory() const { return senderCategory; }
ggg.)        string getSubjectLine() const { return subjectLine; }
hhh.)        string getDateStr() const { return dateStr; }
iii.)
jjj.)        // Comparison operator for MaxHeap:
kkk.)        // Returns true if 'this' email has higher priority than
            'other'.
lll.)        // Priority rules:
mmm.)        // 1. Higher sender category rank comes first.
nnn.)        // 2. If categories are the same, newer date comes first.
ooo.)        bool operator>(const Email& other) const {
ppp.)            if (priorityRank != other.priorityRank) {
qqq.)                return priorityRank > other.priorityRank;
rrr.)            }
sss.)            return dateValue > other.dateValue;
ttt.)        }
uuu.)
vvv.)        bool operator<(const Email& other) const {
www.)            if (priorityRank != other.priorityRank) {
xxx.)                return priorityRank < other.priorityRank;
yyy.)            }
zzz.)            return dateValue < other.dateValue;
aaaa.)        }
bbbb.)    };
cccc.)
dddd.)        // Class representing a List-based MaxHeap from scratch
eeee.)        class MaxHeap {
ffff.)        private:
gggg.)            vector<Email> heap;
hhhh.)
iiii.)        void siftUp(int index) {
jjjj.)            while (index > 0) {
kkkk.)                int parent = (index - 1) / 2;
llll.)                if (heap[index] > heap[parent]) {
mmmm.)                    Email temp = heap[index];
nnnn.)                    heap[index] = heap[parent];
oooo.)                    heap[parent] = temp;
pppp.)                    index = parent;

```

```

qqqq.)          } else {
rrrr.)              break;
ssss.)          }
tttt.)      }
uuuu.)      }
vvvv.)
www.)      void siftDown(int index) {
xxxx.)          int size = heap.size();
yyyy.)          while (2 * index + 1 < size) {
zzzz.)              int leftChild = 2 * index + 1;
aaaa.)              int rightChild = 2 * index + 2;
bbbb.)              int largest = index;
cccc.)
dddd.)              if (leftChild < size && heap[leftChild] > heap[largest])
{
eeee.)                  largest = leftChild;
ffff.)              }
gggg.)              if (rightChild < size && heap[rightChild] >
heap[largest]) {
hhhh.)                  largest = rightChild;
iiii.)              }
jjjj.)
kkkk.)              if (largest != index) {
llll.)                  Email temp = heap[index];
mmmm.)                  heap[index] = heap[largest];
nnnn.)                  heap[largest] = temp;
oooo.)                  index = largest;
pppp.)              } else {
qqqq.)                  break;
rrrr.)              }
ssss.)          }
tttt.)      }
uuuu.)
vvvv.)      public:
www.)          MaxHeap() {}
xxxx.)
yyyy.)          void insert(const Email& email) {
zzzz.)              heap.push_back(email);
aaaa.)              siftUp(heap.size() - 1);
bbbb.)          }
cccc.)
dddd.)          bool isEmpty() const {
eeee.)              return heap.empty();
ffff.)          }
gggg.)

```

```

hhhhh.)      int size() const {
iiiiii.)          return heap.size();
jjjjjj.)      }
kkkkkk.)
llllll.)      Email peek() const {
mmmmmm.)          if (!isEmpty()) {
nnnnnn.)              return heap[0];
oooooo.)          }
pppppp.)          return Email();
qqqqqq.)      }
rrrrrr.)
ssssss.)      void removeMax() {
tttttt.)          if (isEmpty()) return;
uuuuuu.)          heap[0] = heap.back();
vvvvvv.)          heap.pop_back();
wwwwww.)          if (!isEmpty()) {
xxxxxx.)              siftDown(0);
yyyyyy.)          }
zzzzzz.)      }
aaaaaa.) };
bbbbbb.)
ccccc.) // Main CEO Email Management System Object
ddddd.) class CEOInboxSystem {
eeeeee.) private:
ffffff.)     MaxHeap emailHeap;
gggggg.)
hhhhh.) public:
iiiiii.)     void addEmail(string category, string subject, string date) {
jjjjjj.)         Email newEmail(category, subject, date);
kkkkkk.)         emailHeap.insert(newEmail);
llllll.)     }
mmmmmm.)
nnnnnn.)     void displayCount() const {
oooooo.)         cout << "There are " << emailHeap.size() << " emails to
          read." << endl;
pppppp.)     }
qqqqqq.)
rrrrrr.)     void displayNext() const {
ssssss.)         if (emailHeap.isEmpty()) {
tttttt.)             cout << "No emails in the inbox." << endl;
uuuuuu.)             return;
vvvvvv.)         }
wwwwww.)         Email nextEmail = emailHeap.peek();
xxxxxx.)         cout << "Next email:" << endl;

```

```

yyyyyyy.)      cout << "      Sender: " << nextEmail.getSenderCategory() <<
endl;
zzzzzzzz.)      cout << "      Subject: " << nextEmail.getSubjectLine() <<
endl;
aaaaaaaa.)      cout << "      Date: " << nextEmail.getDateStr() <<
endl;
bbbbbbbbb.)      }
cccccccc.)
dddddddd.)      void readEmail() {
eeeeeeee.)          if (!emailHeap.isEmpty()) {
fffffffff.)              emailHeap.removeMax();
ggggggggg.)          }
hhhhhhhhh.)      }
iiiiiii.)      };
jjjjjjjj.)
kkkkkkkk.)      // Driver code to parse command inputs
llllllll.)      int main() {
mmmmmmmm.)          CEOInboxSystem inbox;
nnnnnnnn.)
oooooooo.)      // Example simulation commands matching the sample test
file
pppppppp.)      // In a real file-reading implementation, you would read
line by line using ifstream.
qqqqqqqq.)      string commands[] = {
rrrrrrrr.)          "EMAIL Peer,Can you help me on this?,12-01-2024",
ssssssss.)          "EMAIL OtherPerson,Try our product,12-19-2024",
tttttttt.)          "EMAIL Boss,Important,12-20-2024",
uuuuuuuu.)          "EMAIL Subordinate,How do I handle this?,12-25-2024",
vvvvvvvv.)          "EMAIL ImportantPerson,Health Insurance Enrollment,12-
31-2024",
wwwwwwww.)          "EMAIL Boss,Never Mind,01-03-2025",
xxxxxxx.)          "COUNT",
yyyyyyyy.)          "NEXT",
zzzzzzzz.)          "READ",
aaaaaaaa.)          "NEXT",
bbbbbbbbb.)          "READ",
cccccccc.)          "COUNT"
dddddddd.)      };
eeeeeeee.)
fffffffff.)      for (const string& cmd : commands) {
ggggggggg.)          if (cmd.rfind("EMAIL", 0) == 0) {
hhhhhhhhh.)              // Parse comma-delimited fields: EMAIL
<category>,<subject>,<date>
iiiiiii.)          size_t firstComma = cmd.find(',');

```

```

jjjjjjjjj.)          size_t secondComma = cmd.find(',', firstComma +
1);
kkkkkkkkk.)
lllllllll.)          string category = cmd.substr(6, firstComma - 6);
mmmmmmmmm.)          string subject = cmd.substr(firstComma + 1,
secondComma - firstComma - 1);
nnnnnnnnn.)          string date = cmd.substr(secondComma + 1);
ooooooooo.)
ppppppppp.)          inbox.addEmail(category, subject, date);
qqqqqqqqq.)          } else if (cmd == "COUNT") {
rrrrrrrrr.)          inbox.displayCount();
sssssssss.)          } else if (cmd == "NEXT") {
ttttttttt.)          inbox.displayNext();
uuuuuuuuu.)          } else if (cmd == "READ") {
vvvvvvvvv.)          inbox.readEmail();
wwwwwwwww.)          }
xxxxxxxxx.)          }
yyyyyyyyy.)
zzzzzzzzz.)          return 0;
aaaaaaaaa.)          }

```

This next one is chatgpt's

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <sstream>
```

```
using namespace std;
```

```
// -----
```

```
// Email class
```

```
// -----
```

```
class Email
```

```
{
```

private:

string sender;

string subject;

string date;

public:

Email()

{

sender = "";

subject = "";

date = "";

}

Email(string s, string sub, string d)

{

sender = s;

subject = sub;

date = d;

}

string getSender()

{

***return* sender;**

}

string getSubject()

```
{  
    return subject;  
}
```

```
string getDate()  
{  
    return date;  
}
```

```
int getPriority()  
{  
    if (sender == "Boss")  
        return 5;  
    else if (sender == "Subordinate")  
        return 4;  
    else if (sender == "Peer")  
        return 3;  
    else if (sender == "ImportantPerson")  
        return 2;  
    else  
        return 1;  
}
```

```
// Convert MM-DD-YYYY into YYYYMMDD
```

```
// This makes date comparisons easy.
```

```
int getDateValue()
```

```

{
    int month = stoi(date.substr(0, 2));
    int day = stoi(date.substr(3, 2));
    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
}

};

// -----

// Node class

// -----

class HeapNode
{
public:
    Email email;
    HeapNode* next;

    HeapNode(Email e)
    {
        email = e;
        next = nullptr;
    }
};

// -----

```

```

// MaxHeap class
// -----

class MaxHeap
{
private:
    HeapNode* head;
    int size;

    // Return the node at a particular position.
    // Position 1 = root
    HeapNode* getNode(int position)
    {
        HeapNode* current = head;

        for (int i = 1; i < position && current != nullptr; i++)
        {
            current = current->next;
        }

        return current;
    }

    // Determines whether email1 has higher priority
    // than email2.
    bool higherPriority(Email email1, Email email2)
    {

```

```
if (email1.getPriority() > email2.getPriority())
```

```
{
```

```
    return true;
```

```
}
```

```
if (email1.getPriority() < email2.getPriority())
```

```
{
```

```
    return false;
```

```
}
```

```
// Same sender category:
```

```
// newer date gets priority.
```

```
    return email1.getDateValue() > email2.getDateValue();
```

```
}
```

```
void swapEmails(HeapNode* first, HeapNode* second)
```

```
{
```

```
    Email temp = first->email;
```

```
    first->email = second->email;
```

```
    second->email = temp;
```

```
}
```

```
// Move a newly inserted node upward.
```

```
void heapifyUp(int position)
```

```
{
```

```
    while (position > 1)
```

```

{
    int parentPosition = position / 2;

    HeapNode* current = getNode(position);
    HeapNode* parent = getNode(parentPosition);

    if (higherPriority(current->email, parent->email))
    {
        swapEmails(current, parent);
        position = parentPosition;
    }
    else
    {
        break;
    }
}
}

```

// Move the root downward after removing it.

```

void heapifyDown()
{
    int position = 1;

    while (true)
    {
        int leftPosition = position * 2;

```

```
int rightPosition = position * 2 + 1;
```

```
if (leftPosition > size)
```

```
{
```

```
    break;
```

```
}
```

```
int largerChildPosition = leftPosition;
```

```
HeapNode* leftChild = getNode(leftPosition);
```

```
if (rightPosition <= size)
```

```
{
```

```
    HeapNode* rightChild = getNode(rightPosition);
```

```
    if (higherPriority(rightChild->email,
```

```
        leftChild->email))
```

```
    {
```

```
        largerChildPosition = rightPosition;
```

```
    }
```

```
}
```

```
HeapNode* current = getNode(position);
```

```
HeapNode* largerChild = getNode(largerChildPosition);
```

```
if (higherPriority(largerChild->email,
```

```

        current->email))
    {
        swapEmails(current, largerChild);
        position = largerChildPosition;
    }
    else
    {
        break;
    }
}
}

```

public:

MaxHeap()

```

{
    head = nullptr;
    size = 0;
}

```

~MaxHeap()

```

{
    while (head != nullptr)
    {
        HeapNode* temp = head;
        head = head->next;
    }
}

```

```
        delete temp;
    }
}
```

```
void insert(Email email)
```

```
{
    HeapNode* newNode = new HeapNode(email);

    if (head == nullptr)
    {
        head = newNode;
        size = 1;
        return;
    }
```

```
// Add to the end of the linked list.
```

```
HeapNode* current = head;
```

```
while (current->next != nullptr)
```

```
{
    current = current->next;
}
```

```
current->next = newNode;
```

```
size++;
```

```
    heapifyUp(size);  
}
```

```
Email getMax()  
{  
    return head->email;  
}
```

```
void removeMax()  
{  
    if (head == nullptr)  
    {  
        return;  
    }
```

```
// If there is only one email.
```

```
if (size == 1)  
{  
    delete head;  
    head = nullptr;  
    size = 0;  
    return;  
}
```

```
// Find the last node.
```

```
HeapNode* previous = nullptr;
```

```
HeapNode* current = head;
```

```
while (current->next != nullptr)
```

```
{
```

```
    previous = current;
```

```
    current = current->next;
```

```
}
```

```
// Move last email to root.
```

```
head->email = current->email;
```

```
// Remove last node.
```

```
previous->next = nullptr;
```

```
delete current;
```

```
size--;
```

```
heapifyDown();
```

```
}
```

```
int getSize()
```

```
{
```

```
    return size;
```

```
}
```

```
bool isEmpty()
```

```

    {
        return size == 0;
    }
};

// -----
// CEO Inbox class
// -----

class CEOInbox
{
private:
    MaxHeap inbox;

public:

    void receiveEmail(string sender, string subject, string date)
    {
        Email email(sender, subject, date);
        inbox.insert(email);
    }

    void nextEmail()
    {
        if (inbox.isEmpty())
        {
            return;

```

```
}
```

```
Email email = inbox.getMax();
```

```
cout << "Next email:" << endl;
```

```
cout << "  Sender: " << email.getSender() << endl;
```

```
cout << "  Subject: " << email.getSubject() << endl;
```

```
cout << "  Date: " << email.getDate() << endl;
```

```
cout << endl;
```

```
}
```

```
void readEmail()
```

```
{
```

```
    if (!inbox.isEmpty())
```

```
    {
```

```
        inbox.removeMax();
```

```
    }
```

```
}
```

```
void countEmails()
```

```
{
```

```
    cout << "There are " << inbox.getSize()
```

```
        << " emails to read." << endl;
```

```
    cout << endl;
```

```
}
```

```
};
```

```
// -----  
  
// Main  
  
// -----  
  
int main()  
{  
    ifstream inputFile("test.txt");  
  
    if (!inputFile)  
    {  
        cout << "Error opening test file." << endl;  
        return 1;  
    }  
  
    CEOInbox inbox;  
  
    string line;  
  
    while (getline(inputFile, line))  
    {  
        if (line.empty())  
        {  
            continue;  
        }  
  
        // -----
```

```

// EMAIL command

// -----

if (line.substr(0, 5) == "EMAIL")
{
    // Remove "EMAIL "

    string emailData = line.substr(6);

    stringstream ss(emailData);

    string sender;
    string subject;
    string date;

    // Sender
    getline(ss, sender, ',');

    // Subject
    getline(ss, subject, ',');

    // Date
    getline(ss, date);

    inbox.receiveEmail(sender, subject, date);
}

// -----

```

```
// NEXT command

// -----

else if (line == "NEXT")
{
    inbox.nextEmail();
}

// -----

// READ command

// -----

else if (line == "READ")
{
    inbox.readEmail();
}

// -----

// COUNT command

// -----

else if (line == "COUNT")
{
    inbox.countEmails();
}
}

inputFile.close();
```

```
return 0;
```

```
}
```